Learning Guides: os-midterm

Compiled on 2026-03-18 14:34:19

Total documents: 1

These are simplified learning guides. Use these for studying instead of the lengthy originals.

Table of Contents

- Learning Guide: os-midterms.pdf

Learning Guide: os-midterms.pdf

Generated on 2026-03-18 14:34:19

This is a simplified learning guide created from the original PDF. Use this for studying instead of reading the lengthy original text.

Pages 1-20

Here is a simplified, easy-to-read learning guide based on the provided text:

Operating System Fundamentals: Study Guide

1. Introduction to Operating Systems (OS)

- **What is an Operating System (OS)?**
  - A program that acts as an intermediary (bridge) between a computer user and the computer hardware.
- **Goals of an Operating System:**
  1. To execute user programs and make solving user problems easier.
  2. To make the computer system convenient and easy to use.
  3. To use the computer hardware in an efficient manner.

2. Computer System Components

A computer system consists of four main components:

1. **Hardware:** Provides the basic computing resources.
   - **Examples:** CPU (Central Processing Unit), Memory (RAM), I/O devices (Input/Output).
2. **Operating System (OS):**
   - Controls and coordinates the use of the hardware among various application programs and users.
3. **Application Programs:**
   - Define how system resources are used to solve user computing problems.
   - **Examples:** Compilers, database systems, video games, web browsers, word processors.
4. **Users:**
   - People, machines, or other computers interacting with the system.

Interaction Flow: Users interact with Application Programs, which use the Operating System to access and manage the Computer Hardware.

3. Computer System Booting Sequence

The process a computer follows when it starts up:

1. **Bootstrap Program:**
    - The initial program loaded when the computer starts.
    - Stored in the computer hardware's **firmware** (ROM or EEPROM).
    - Its job is to initialize the basic hardware needed to start the OS.
2. **Kernel:**
    - The core part of the Operating System.
    - It acts as a bridge between software applications and the hardware.
    - Initializes essential system resources:
        - **CPU scheduler:** Manages CPU time for different tasks.
        - **Memory manager:** Handles how memory is used.
        - **Device drivers:** Allow the OS to communicate with hardware devices.
3. **System Daemons:**
    - Programs that run continuously in the background.
    - Operate in "user space" (meaning they don't have direct hardware access like the kernel).
    - Provide specific services without user interaction (e.g., networking services, logging).

Once all these steps are complete, the Operating System is fully booted and ready to respond to user events.

4. Computer System Storage Structures

Computer storage is organized in a hierarchy based on speed, cost, and capacity.

- **The Speed Gap:** CPUs are incredibly fast, but large storage devices are relatively slow. This creates a "speed gap" that needs to be managed for efficient operation.
- **Storage Characteristics:**
    - **Higher in Hierarchy = Faster** access speed.
    - **Higher in Hierarchy = More expensive** per bit.

- **Volatile:** Loses its content when power is turned off (e.g., Registers, Cache, RAM).
- **Nonvolatile:** Retains its content even when power is off (e.g., SSD, HDD).

Storage Hierarchy (from fastest/smallest to slowest/largest):

1. **CPU Registers:**
   - **Location:** Inside the CPU core.
   - **Speed:** Fastest possible memory.
   - **Size:** Extremely small (bits or bytes).
   - **Purpose:** Holds immediate data and instructions the CPU is actively processing (e.g., `1 + 1`).
   - **Volatility:** Volatile.
2. **Cache Memory (L1, L2, L3):**
   - **Location:** Between CPU Registers and Main Memory. L1 is closest to the CPU, L3 is furthest.
   - **Purpose:** Stores frequently used instructions and data to speed up access.
   - **How it works:** The computer first checks the cache for needed data/instructions. If not found (a "cache miss"), it then looks in Main Memory.
   - **Size:** L1 (~KBs), L2 (~MBs), L3 (~MBs, but larger than L1/L2).
   - **Volatility:** Volatile.
3. **Main Memory (RAM - Random Access Memory):**
   - **Location:** External to the CPU, accessible by the CPU.
   - **Speed:** Much larger than cache but not as fast.
   - **Size:** Typically Gigabytes (~GBs).
   - **Purpose:** Holds all currently running programs and open files. This is where the OS loads programs before they execute.
   - **Volatility:** Volatile.
4. **Secondary Storage (Hard Disk Drive - HDD / Solid State Drive - SSD):**
   - **Location:** Persistent storage devices.
   - **Speed:** Much slower than RAM, but offers large capacity.
   - **Purpose:** Stores the operating system, applications, and user data permanently.
   - **Volatility:** Nonvolatile.

- **Types:**
  - **Solid State Drive (SSD):**
    - No moving parts.
    - Faster than HDDs.
    - More expensive per GB than HDDs.
  - **Hard Disk Drive (HDD):**
    - Uses spinning magnetic platters.
    - Slower than SSDs.
    - Cheaper per GB than SSDs.

Pages 19-38

Here is a simplified, easy-to-read learning guide based on the provided text:

Computer System Learning Guide

1. Computer System Storage Structures

This section explains how data is stored and accessed within a computer system.

1.1. Main Memory (RAM)

- **Size**: Significantly larger than caches (Gigabytes - GBs).
- **Purpose**: Holds all currently running programs and open files.
- **Volatility**: **Volatile** – data is lost when the computer is turned off.
- **Type**: Random Access Memory (RAM).

1.2. Secondary Memories

These are non-volatile storage devices, meaning they retain data even when power is off. They store installed programs and persistent data.

- **Solid State Drive (SSD)**
  - **Mechanism**: No moving parts.
  - **Speed**: Fast.
  - **Cost**: More expensive.
  - **Volatility**: Non-volatile.
- **Hard Disk Drive (HDD)**
  - **Mechanism**: Uses spinning magnetic platters.
  - **Speed**: Slower.
  - **Cost**: Cheaper.
  - **Volatility**: Non-volatile.

1.3. Data Flow and Execution

1. **Program Storage**: Installed programs reside on **Secondary Memories** (HDD/SSD).
2. **Program Execution**: When a program is run, it is copied from **Secondary Memory** to **Main Memory (RAM)**.
3. **Computation**: The **CPU CORE** (Central Processing Unit) performs computations, using its **Registers** (e.g., RAX, RBX) for very fast temporary storage.
4. **CPU Data Fetch**: The CPU doesn't directly access Main Memory. All CPU data fetches are intercepted by **Caches**.
   - **Caches**: Small, extremely fast memory banks closer to the CPU.
     - **L1 Cache**: Smallest, fastest, closest to the CPU.
     - **L2 Cache**: Larger than L1 (~Kilobytes - KBs).
     - **L3 Cache**: Largest of the caches (~Megabytes - MBs).
   - **Hierarchy**: CPU <-> Cache (L1, L2, L3) <-> Main Memory (RAM) <-> Secondary Memory (HDD/SSD).

---

2. Computer System I/O Structure

This section covers how input/output operations are managed within a computer.

2.1. Key Components

- **Device Controllers**:
  - **Nature**: Hardware.
  - **Role**: Responsible for moving data between peripheral devices (e.g., keyboard, mouse, disk drive) and its own local buffer storage.
- **Device Drivers**:
  - **Nature**: Software, part of the Operating System.
  - **Role**: Provides a uniform interface, allowing the OS to interact with different hardware devices without needing to know their specific details.

2.2. Standard I/O Operation (for small data)

1. The **device driver** loads appropriate commands into registers within the **device controller**.
2. The **device controller** reads these registers to understand the required action.
3. The controller starts transferring data from the device to its local buffer.
4. Once finished, the device controller informs the device driver via an **interrupt**.
   - **Interrupt**: A signal from hardware to the CPU, indicating that an event has occurred and needs attention.
5. The device driver then returns control to the operating system, providing data (if it was a read operation) or status information.

2.3. Direct Memory Access (DMA) (for large data)

Standard I/O is inefficient for large amounts of data due to frequent CPU involvement and interrupts. DMA solves this.

- **Mechanism**: After initial setup (buffers, pointers, counters) by the CPU, the **device controller** can transfer entire blocks of data directly between its own buffer storage and main memory.
- **CPU Involvement**: No intervention by the CPU is required during the data transfer itself.
- **Interrupts**: Only one interrupt is generated by the controller to signal the CPU *after* the entire large data block transfer is complete.

---

3. Computer System Architecture

This section discusses how computer systems are structured, particularly concerning fault handling and processor configurations.

3.1. Fault Handling

Mechanisms to deal with errors and failures in a system.

- **Graceful Degradation**:
  - **Concept**: The system continues to provide service, but its performance is reduced proportionally to the level of surviving hardware.
  - **Outcome**: System remains usable, but with decreased performance.
  - *Example*: If some RAM fails, the system might run slower or restrict certain applications, but doesn't crash entirely.
- **Soft Failure**:
  - **Concept**: The system can detect and isolate errors without a total crash.
  - **Outcome**: The faulty component or process is isolated, and the system continues running, albeit potentially with reduced functionality related to the isolated part.

- **Fault Tolerant**:
    - **Concept**: The system can continue fully operating even if a single component fails. It can seamlessly work around the failure.
    - **Outcome**: System continues normal operation despite component failure, often by having redundant components.

3.2. System Types

Different ways computer processors are configured.

- **Single-Processor Systems**
    - **Configuration**: Contains only one general-purpose CPU.
    - **Support**: May work with specialized microprocessors for specific tasks (e.g., disk, keyboard, graphics processors).
- **Multiprocessor Systems** (aka Parallel Systems or Multicore Systems)
    - **Configuration**: Has two or more general-purpose CPUs.
    - **Memory Access**: Changes the memory access model from **Uniform Memory Access (UMA)** to **Non-Uniform Memory Access (NUMA)**.
        - **UMA**: All CPUs have equal access time to all parts of main memory.
        - **NUMA**: Access time to memory varies depending on which CPU is accessing which part of memory (e.g., a CPU accessing local memory is faster than accessing memory attached to another CPU).
    - **Advantages**:
        1. **Increased Throughput**: More work can be completed in the same amount of time.
        2. **Economy of Scale**: Often more cost-effective to add multiple processors to one system than to buy multiple single-processor systems.
        3. **Increased Reliability**: If one CPU fails, other CPUs can potentially take over its tasks, improving system uptime.
    - **Types**:
        - **Asymmetric Multiprocessing**:
            - **Structure**: Consists of a "boss" processor that controls the entire system.

        - **Relationship**: Defines a master-slave or boss-worker relationship where the boss processor assigns tasks to others.
    - **Symmetric Multiprocessing (SMP)**:
        - **Structure**: The most common type. All processors are peers.
        - **Role**: Each processor performs all tasks within the Operating System, sharing the workload equally.
- **Clustered Systems**
    - **Configuration**: A type of multiprocessor system where multiple independent computers (nodes) are linked together.
    - **Coupling**: Loosely coupled (compared to tightly coupled multiprocessor systems where CPUs share memory).
    - **Purpose**: Primarily designed to provide **high-availability** (continuous operation even if one node fails) and often increased computing power.

Pages 37-56

Here is a simplified, easy-to-read learning guide based on the provided text:

Computer System Architecture & Operating Systems: Learning Guide

I. Computer System Architecture

A. Multiprocessor Systems

- **Definition:** Systems with two or more general-purpose CPUs (also called Parallel Systems or Multicore Systems).
- **Memory Access:** Changes from Uniform Memory Access (UMA) to Non-Uniform Memory Access (NUMA).
- **Advantages:**
  - Increased throughput (more tasks per unit time).
  - Economy of scale (cost-effective).
  - Increased reliability (if one CPU fails, others can continue).
- **Types:**
  - **Asymmetric Multiprocessing:** Has a "Boss" processor controlling the system (master-slave relationship).
  - **Symmetric Multiprocessing (SMP):** Most common type. All processors are peers and perform all OS tasks.
- **Multicore Chips:**
  - Multiple computing cores on a single chip.
  - Faster and use less power than multiple single-core chips.

B. Clustered Systems

- **Definition:** A type of multiprocessor system that is "loosely coupled." Designed for **high-availability** through **redundancy**.
- **Structure:** Composed of two or more independent computer systems (nodes) joined together.
- **Failover Mechanism:** Each node monitors others. If a node fails, a monitoring node takes over its tasks and resources, allowing the failed node to restart.

- **Types:**
  - **Symmetric:** All machines run applications and can run more than one application.
  - **Asymmetric:** One machine is in "hot-standby" mode, ready to take over if the active node fails.
- **Capability:** Can provide **high-performance computing** (greater than SMP) through parallelization.

II. Operating System (OS) Fundamentals

A. Purpose of an Operating System

- **Manages hardware resources** and provides services.
- Enables multiple programs to **execute efficiently and safely**.
- **Key Responsibilities:**
  - Managing execution (processes, threads).
  - Managing resources (CPU, memory, I/O).
  - Coordinating concurrent activities.
  - Sharing the system among users and programs.

B. Core OS Concepts

1. **Process:**
   - A self-contained execution environment.
   - Has a complete, private set of basic runtime resources, including its own memory space.
2. **Thread:**
   - The smallest unit of CPU execution *within* a process.
   - Threads within the same process share basic runtime resources (like memory space).
3. **Interrupt:**
   - A signal that indicates an event has occurred (from hardware or software).

    - Requires immediate attention from the CPU.
    - **Action:** Temporarily pauses the current CPU task to handle the event (via an Interrupt Service Routine), then resumes the interrupted task.

III. Types of Operating Systems

A. Batch Operating Systems

- **Function:** Processes similar jobs in groups (batches) without user interaction during execution.
- **Key Innovation: Multiprogramming**
    - **Definition:** Improves CPU **utilization** by organizing jobs (code and data) so the CPU is always busy.
    - Keeps multiple processes in memory simultaneously.
    - When one process waits for I/O, the CPU switches to another ready job to avoid idle time.
- **Key Defining Feature: Utilization** (of the CPU).
- **Nature:** Not interactive.

B. Time-Sharing Operating Systems

- **Concept:** A logical extension of Multiprogramming.
- **Function:** The CPU executes multiple processes by rapidly switching among them. The switches are so frequent that users can **interact** with each program while it's running.
- Allows multiple users to share the system concurrently.
- **Mechanism:**
    - CPU time is divided into small units called **time slices (or quantum)**.
    - Each process gets the CPU for one time slice.
    - A **timer interrupt** enforces these time slices, forcing the CPU to switch processes.
    - An **input interrupt** allows responsiveness to user actions.
- **Key Defining Feature: Responsiveness** (to user interaction).

Pages 55-74

Here is a simplified, easy-to-read learning guide based on the provided text:

Operating Systems Learning Guide (Pages 55-74)

I. Operating Systems Fundamentals

A. Time-Sharing Operating Systems

Time-sharing is a logical extension of multiprogramming that allows multiple users to interact with a single computer system simultaneously.

1. **Core Concept:**
    - The CPU executes multiple processes by rapidly switching between them.
    - Switches occur so frequently that users can interact with each program as if it were running continuously.
    - Allows multiple users to share one system.
2. **Key Mechanisms:**
    - **CPU Time Division:** CPU time is divided into small units called **time slices** (or **quantum**).
    - **Process Allocation:** Each process gets the CPU for one time slice.
    - **Timer Interrupt:** Enforces time slices, ensuring no single process monopolizes the CPU.
    - **Input Interrupt:** Allows the system to be responsive to user actions (e.g., keyboard input, mouse clicks).

3. **Key Defining Feature:**
    - **Responsiveness:** The system quickly reacts to user input and provides timely feedback.

B. User and Operating System Interface

Almost all operating systems provide a **User Interface (UI)** to allow interaction.

1. **Types of User Interfaces (UI):**
    - **Command Line Interface (CLI):** Also known as a **Command Interpreter (CI)**. Users type commands directly.
    - **Graphical User Interface (GUI):** A user-friendly interface using visual elements (icons, windows) and input devices (mouse, keyboard).
    - **Batch:** Commands are entered into files, which are then run collectively.
2. **Command Line Interface (CLI) / Command Interpreter (CI):**
    - **Shells:** On systems with multiple Command Interpreters, they are known as Shells (e.g., Bash in Linux).
    - **Shell Scripts:** A set of command-line steps saved in a file that can be executed like a program.
    - **Common CLI Commands:** Often used for file manipulation (e.g., create, delete, list, print, copy, execute).
3. **CLI Approaches:**
    - **Interpreter Implements Commands:** The code for the command (e.g., `cd` for 'change directory' in Windows cmd) is part of the interpreter itself.
    - **Program Implements Commands:** The command (e.g., `rm` for 'remove' in UNIX/Linux) is a separate program that the interpreter executes with parameters.

II. Interacting with the OS

A. System Calls

System Calls are a mechanism used by programs to request services from the operating system (OS).

1. **Definition & Purpose:**
    - Allow user-mode programs to access kernel-mode services (e.g., file I/O, process creation, memory management).
    - Enable interaction with hardware devices.
2. **Application Programming Interface (API):**
    - Specifies a set of functions available to application programmers.
    - **System Call Interface:** Acts as a link, intercepting API function calls and invoking the necessary system calls provided by the OS (which are often stored in a table).
    - **Examples:** Windows API, POSIX API (for UNIX, Linux, macOS), Java API.
3. **Passing Parameters to the OS:**
    - **Via Registers:** Parameters are stored directly in CPU registers. Limited by the number of available registers.
    - **Via Address:** Parameters are stored in a block of memory, and the memory block's address is passed via a register.
    - **Via Stack:** Parameters are pushed onto the process's stack by the program and then popped off by the operating system.
4. **Types of System Calls:**
    - **Process Control:** Manages process creation, termination, and synchronization (e.g., `fork()`, `exit()`).
    - **File Manipulation:** Handles operations on files (e.g., `open()`, `read()`, `write()`, `delete()`).
    - **Device Manipulation:** Manages interaction with hardware devices (e.g., `ioctl()` for I/O control, `read()`, `write()`).

- **Information Maintenance:** Retrieves and modifies system data (e.g., `getpid()` for process ID, `settimeofday()` for system time).
- **Communication:** Enables inter-process communication (IPC) via message passing or shared memory (e.g., `pipe()`, `socket()`).
- **Protection:** Controls resource access and enforces security policies (e.g., `chmod()` for file permissions, `setuid()` for user ID).

B. The Shell

The **Shell** is the outermost layer of the OS, acting as a special user program that provides an interface for users to access operating system services.

1. **Definition:**
   - It's the command interpreter that processes commands entered by the user.
2. **Shell Scripts:**
   - A way to input multiple commands via a file, automating tasks.
3. **Common Linux Shells:**
   - **BASH** (Bourne Again SHell)
   - **CSH** (C SHell)
   - **KSH** (Korn SHell)
4. **Command Prompt Symbols in CLI:**
   - `$` : Indicates the current user is a **normal user**.
   - `#` : Indicates the current user is a **root user** (superuser) with administrative privileges.

III. Virtualization Basics

A. Hypervisor (Virtual Machine Monitor - VMM)

A **Hypervisor** is a program that creates and runs **virtual machines (VMs)**, allowing multiple operating systems to share a single hardware platform.

1. **Definition:**
    - Also known as a Virtual Machine Monitor (VMM).
    - Manages and isolates guest operating systems from the underlying hardware.
2. **Types of Hypervisors:**
    - **Type I (Bare-Metal/Native):**
        - Runs directly on the host hardware.
        - Acts like a lightweight operating system itself.
        - Examples: VMware ESXi, Microsoft Hyper-V, Xen.
    - **Type II (Hosted):**
        - Runs on top of an existing host operating system (e.g., Windows, macOS, Linux).
        - Borrows resources from the host OS.
        - Examples: VMware Workstation, Oracle VirtualBox.

Pages 73-92

Here's a simplified, easy-to-read learning guide based on the provided text:

Linux Command Line Basics & Shell Scripting Guide

This guide covers essential Linux command-line interface (CLI) concepts, file management, process control, and an introduction to shell scripting.

1. The Shell

The **Shell** is the outermost layer of the operating system, acting as a special user program that provides an interface for you to access OS services.

- **Function**: It takes your commands and executes them by interacting with the OS kernel.
- **Shell Scripts**: Files containing a sequence of shell commands for automated execution.
- **Common Linux Shells**:
  - **BASH** (Bourne Again SHell) - Most common.
  - **CSH** (C SHell)
  - **KSH** (Korn SHell)

2. Working with the Command Line Interface (CLI)

The CLI is a text-based interface to interact with your system.

2.1. Prompt Symbols

The symbol preceding your command prompt indicates your user type: * `$` : Normal user * `#` : Root user (superuser)

2.2. User and Hostname Indicator

The prompt often shows `username@hostname`, indicating: * `username`: The current user logged in. * `hostname`: The name of the device you are logged into.

2.3. Special Directories

- `~` (Tilde): Represents the current user's **home directory** (e.g., `/home/username/`).
- `/` (Slash): Represents the **root directory**, the top-level directory in the file system hierarchy.

2.4. Normal User vs. Root User

Feature	Normal User	Root User (Superuser)
Access	Restricted; operates within `/home/username/` and specific system areas.	Unrestricted access to *all* system resources and files.
Permissions	Can only modify their own files and run software without system-wide changes.	Can modify critical system files and perform system-wide changes.
Password	Can change only their own password.	Can change any user's password.
Admin Rights	Can use `sudo` (if allowed) to execute specific commands with root privileges temporarily.	Has full administrative rights by default.

2.5. Checking Current User

- `whoami`: Prints the current user's username.
  - Example: `$ whoami` -> `mario`
- `id -u`: Prints the current user's numeric User ID (UID).
  - **Root ID is always `0`**.
  - Example: `$ id -u` -> `1000` (for a normal user); `0` (for root).

2.6. Identifying & Changing Your Shell

- `which $SHELL`: Shows the full path of your current shell (e.g., `/bin/bash`).
- `chsh`: Change your default login shell.
  - `$ chsh`: Prompts for the new shell path.
  - `$ chsh -s /bin/bash`: Directly specifies `/bin/bash` as the new shell.

3. File and Directory Management

These commands help you navigate and manage files and directories.

Command	Description	Common Options
`pwd`	**P**rints **W**orking **D**irectory - Displays the current directory path.	(None)
`ls`	**Li**s**t**s files and directories in the current path.	`-l`: Long format (details: permissions, owner, size) `-a`: Show **a**ll files, including hidden ones (start with `.`) `-h`: **H**uman-readable sizes (e.g., `11K` instead of `11264`) `-R`: **R**ecursive, lists contents of subdirectories. *(Combine options, e.g., `ls -lah`)*
`cd dir`	**C**hange **D**irectory - Moves to `dir`. *Use `~` for home, `/` for root.*	(None)
`mkdir my_folder`	**M**a**k**e **Dir**ectory - Creates a new directory named `my_folder`.	`-p`: Creates parent directories if they don't exist.
`rmdir my_folder`	**R**e**m**ove **Dir**ectory - Deletes an *empty* directory.	(None)

Command	Description	Common Options
`rm file/dir`	**R**e**m**ove - Deletes files or directories.	`-r` : **R**ecursive, removes directories and their contents. `-f` : **F**orce delete without confirmation. `-i` : **I**nteractive, asks before each deletion. `-v` : **V**erbose, shows deleted files.
`cp src_file dest`	**C**o**p**y - Copies `src_file` to `dest`.	`-r` : **R**ecursive, copies directories and their contents. `-i` : **I**nteractive, prompts before overwriting. `-u` : **U**pdate, copy only if source is newer or destination is missing. `-v` : **V**erbose, shows progress.
`mv src_file dest`	**M**o**v**e or **R**ename - Moves `src_file` to `dest` or renames it.	`-i` : **I**nteractive, asks before overwriting. `-u` : **U**pdate, move only if source is newer or destination is missing. `-v` : **V**erbose, shows progress.

3.1. Understanding `ls -l` Output

Example: `-rwxrw-r-- 3 testuser usergroup 11K Feb 14 19:30 file.txt`

Part	Description	Example Value
File Type	First character: `-` for a regular file, `d` for a directory.	`-`
Permissions	Nine characters in three sets (`rwx`, `rw-`, `r--`) for **owner**, **group**, and **others** respectively.	`rwxrw-r--`

Part	Description	Example Value
	`r`=read, `w`=write, `x`=execute, `-`=no permission.	
Hard Links	Number of hard links to the file (for files) or subdirectories (for directories, including `.` and `..`).	`3`
Owner	The user who owns the file/directory.	`testuser`
Group	The group that owns the file/directory.	`usergroup`
File Size	Size of the file (human-readable with `-h` option).	`11K`
Date Modified	Last modification date and time.	`Feb 14 19:30`
File Name	Name of the file or directory.	`file.txt`

4. File Viewing and Editing

Command	Description	Navigation/Shortcuts
`cat file.txt`	Concatenate - Displays the entire content of `file.txt` to the screen.	(None)
`less file.txt`	Displays `file.txt` content page by page, allowing scrolling. Useful for large files.	`SPACE`: Scroll down `b`: Scroll up `/keyword`: Search for `keyword` `q`: Quit
`nano file.txt`	Opens `file.txt` in the Nano text editor. Simple and user-friendly.	`CTRL-X`: Exit (prompts to save if changes made) `CTRL-O`: Save changes `CTRL-W`: Search

5. File Permissions & Ownership

Command	Description
`chmod 755 script.sh`	**Ch**ange **Mod**e - Modifies file permissions. **Numeric Mode**: Uses three digits (0-7) representing permissions for **Owner**, **Group**, and **Others**. • `4` = Read (`r`) • `2` = Write (`w`) • `1` = Execute (`x`) • `0` = No permission (`-`) *Example `755`: Owner has `rwx` (4+2+1=7), Group has `rx` (4+1=5), Others have `rx` (4+1=5).* **Symbolic Mode**: `+w` (adds write permission), `-w` (removes write permission), `u+x` (add execute for user).
`chown user:group file.txt`	**Ch**ange **Own**er - Changes the owner user and/or group of `file.txt`. *Example `chown jefferson:devs project.txt` makes `jefferson` the owner and `devs` the group owner.*

6. Process Management

These commands allow you to monitor and control running programs (processes).

Command	Description	Navigation/Options
`ps aux`	**P**rocess **S**tatus - Lists currently running processes.	`a` : Show processes for **a**ll users `u` : Show detailed **u**ser information `x` : Show processes not associated with a terminal
`top`		`q` : Quit `k` : **K**ill a process (prompts for PID)

Command	Description	Navigation/Options
	Shows real-time system statistics (CPU, memory, running processes).	`M`: Sort by **M**emory usage
`kill PID`	Terminates a process identified by its **P**rocess **ID** (`PID`).	`-9`: **Force kill** (immediately terminates, no cleanup). `-15`: **Graceful kill** (sends termination signal, allows process to shut down cleanly).

7. Shell Scripts

A **Shell Script** is a text file containing a series of commands executed sequentially by a shell interpreter. They are used to automate tasks.

7.1. Key Components of a Shell Script

Here's an example script and its components:

```
#!/bin/bash
# This script prints "Hello, World!"

echo "Hello, World!"
echo "Listing files:"
ls -l

name="Jefferson"
echo "Hello, $name!"

echo "Enter your name:"
read user_name
echo "Hello, $user_name!"

if [ "$name" == "Jefferson"  ]; then
  echo "Welcome, $name!"
```

```
else
  echo "Access Denied!"
fi

for i in {1..5}; do
  echo "Iteration $i"
done

count=1
while [ "$count"  -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done

greet() {
  echo "Hello, $1!"
}
greet "Jefferson"

mkdir test_dir
echo "Exit status: $?"
```

1. **Shebang (`#!`)**:
   - **Purpose**: The very first line of a script. It specifies the interpreter (e.g., `/bin/bash`) that should execute the script.
   - **Example**: `#!/bin/bash`
2. **Comments (`#`)**:
   - **Purpose**: Lines starting with `#` are ignored by the shell. Used to add explanations or disable parts of the script.
   - **Example**: `# This script prints "Hello, World!"`

3. **Commands**:
    - **Purpose**: Standard Unix/Linux commands (`ls`, `echo`, `mkdir`, etc.) that the script executes.
    - **Example**: `echo "Hello, World!"`, `ls -l`
4. **Variables**:
    - **Purpose**: Store data (e.g., numbers, text strings). Can be system-defined (like `$SHELL`) or user-defined.
    - **Important**: **Spacing matters!** `name="Jefferson"` is correct. `name = "Jefferson"` is incorrect.
    - **Example**: `name="Jefferson"`, `echo "Hello, $name!"`
5. **Input and Output**:
    - **Purpose**: Interact with the user. `read` takes input, `echo` produces output.
    - **Example**: `read user_name`, `echo "Hello, $user_name!"`
6. **Control Statements** (Implicit in original text, but crucial for scripting):
    - **Purpose**: Allow scripts to make decisions and repeat actions.
    - **Examples**:
        - **`if/else`**: Conditional execution. `bash if [ "$name" == "Jefferson" ]; then echo "Welcome, $name!" else echo "Access Denied!" fi`
        - **`for` loop**: Iterates over a list of items. `bash for i in {1..5}; do echo "Iteration $i" done`
        - **`while` loop**: Repeats commands as long as a condition is true. `bash count=1 while [ "$count" -le 5 ]; do echo "Count: $count" count=$((count + 1)) done`
7. **Functions** (Implicit in original text):
    - **Purpose**: Group commands into reusable blocks.
    - **Example**: `bash greet() { echo "Hello, $1!" # $1 refers to the first argument passed to the function } greet "Jefferson"`

8. **Exit Status** (Implicit in original text):
    - **Purpose**: Every command returns an exit status. `0` usually indicates success, non-zero indicates an error.
    - **Variable**: `$?` holds the exit status of the *last* executed command.
    - **Example**: `mkdir test_dir; echo "Exit status: $?"` (prints 0 if `test_dir` was created successfully).

Pages 91-110

Here is a simplified, easy-to-read learning guide derived from the provided text.

Shell Scripting Fundamentals

What is a Shell Script?

A **Shell Script** is a text file containing a series of commands executed sequentially by a **shell interpreter**. It automates tasks.

Core Shell Script Components

1. **Shebang (`#!/bin/bash`)**
    - The first line of a script.
    - Specifies the interpreter (e.g., `bash`) to execute the script.
2. **Comments (`#`)**
    - Lines starting with `#` are comments.
    - Used for explanations and ignored by the interpreter.
3. **Basic Output (`echo`)**
    - `echo "Hello, World!"` prints text to the terminal.

4. **Variables**
    - Store data (e.g., strings, numbers).
    - Can be **system-defined** (e.g., `$PATH`) or **user-defined**.
    - **Assignment:** `name="Jefferson"` (NO spaces around `=`).
    - **Access:** `echo "Hello, $name!"` (use `$` to get the value).
5. **Input and Output**
    - **Output:** `echo "Enter your name:"`
    - **Input:** `read user_name` (reads user input into `user_name` variable).
6. **Control Statements** Allow scripts to make decisions and repeat actions.
    - **Conditional (`if-else`)** `bash if [ "$name" == "Jefferson" ]; then echo "Welcome, $name!" else echo "Access Denied!" fi`
        - `if`, `then`, `else`, `fi` should typically be on new lines.
        - **Checking Conditions:** Use `test` command or square brackets `[]`.
            - `test "$name" == "Jefferson"`
            - `[ "$name" == "Jefferson" ]` (spaces inside brackets are crucial).
    - **Comparison Operations:**
        - **Numbers:**
            - `-eq`: equal
            - `-ne`: not equal
            - `-gt`: greater than
            - `-lt`: less than
            - `-ge`: greater or equal
            - `-le`: less or equal
        - **Strings:**
            - `=`: equal
            - `!=`: not equal

        - `==`: (often synonymous with `=` for strings, but `=` is POSIX standard)

    - **Loops (`for`, `while`)**
        - `for i in {1..5}; do echo "Iteration $i"; done`
        - `while [ "$count" -le 5 ]; do echo "Count: $count"; count=$((count + 1)); done`
        - `for`, `do`, `done`, `while` should typically be on new lines. A semicolon `;` can separate commands on a single line if needed.

7. **Functions**
    - Organize reusable blocks of code.
    - **Definition:** `bash greet() { echo "Hello, $1!" }`
        - Opening bracket `{`, body, and closing bracket `}` typically on different lines.
    - **Calling:** `greet "Jefferson"`
    - **Arguments:** `$1` represents the first argument passed to the function, `$2` the second, and so on.

8. **Exit Status (`$?`)**
    - Every command executed returns an **exit status** (an integer).
    - `0` typically indicates success.
    - Any non-zero value indicates an error.
    - `echo $?` prints the exit status of the *last* executed command.

Expansion Operator (`$`)

The `$` operator is used to retrieve values or perform specific operations:

- **`$()`**: **Command Expansion** – Executes a command and substitutes its output. (e.g., `variable=$(ls)`)
- **`$(())`**: **Arithmetic Expansion** – Performs arithmetic calculations. (e.g., `result=$((5 + 5))`)
- **`$name`**: **Variable Expansion** – Returns the actual value of a variable.
- **`$$`**: Returns the **Process ID (PID)** of the current shell.

- `$?`: Returns the **exit status** of the last command.
- `$1`, `$2`, **...**: **Positional Arguments** – Accesses arguments passed to a script or function.

Processes and Threads

Process vs. Thread

- **Process:**
  - A self-contained execution environment.
  - Has its own complete, private set of basic run-time resources, including its **own memory space**.
  - Independent of other processes.
- **Thread:**
  - Also known as a **lightweight process**.
  - Exists *within* a process (every process has at least one thread).
  - Threads within the same process **share** basic run-time resources (e.g., memory space, open files).

Thread Basics (POSIX Threads in C)

- **POSIX Threads (`pthread.h`)**: A standard C library for creating and managing threads.
- **Main/Parent Thread**: Every C program starts with at least one thread, called the main or parent thread.
- **Child Threads**: All other threads created by the program are child threads.

Important Note on Thread Termination:

In C/C++ programs, if the main thread terminates, all other child threads typically terminate as well. Therefore, it's crucial to wait for child threads to complete their execution before the main thread returns, to ensure all tasks are finished.

Why Use Threads? (Concurrent Execution)

Threads enable **concurrent execution**, allowing a program to run multiple tasks seemingly at the same time without interrupting the main program flow.

Example Scenario: Imagine a program that needs to: 1. Read a very long file. 2. Do lots of other long, independent processes. 3. Finally, process the data from the read file (which depends on step 1).

Without Threads (Sequential): The program would execute step 1, then step 2, then step 3. If step 1 and 2 are very long, the total execution time is the sum of all three.

With Threads (Concurrent): * You can start **reading the long file (Step 1)** in one thread. * **Simultaneously**, you can start **other long processes (Step 2)** in another thread (or the main thread). * Since Step 3 depends on Step 1, the main thread can **wait (`join`)** for the "read file" thread to finish before proceeding with Step 3. * This significantly reduces the overall execution time by overlapping independent tasks.

How to Create Threads (Using `pthread.h` in C)

1. **Include the Header:** `#include <pthread.h>`
2. **Define the Thread Function:**
    - This is the function that the new thread will execute.
    - It **must** return `void*`.
    - It **must** accept a `void*` argument. `c void* child_function(void* arg) { printf("Thread is running.\n"); // ... do thread's work ... return NULL; // Return a value, or NULL if not needed }`
3. **Declare a `pthread_t` Variable:**
    - This variable will store the thread ID. `c pthread_t thread_id;`

4. **Create the Thread using `pthread_create()`:**
    - This function starts a new thread. `c pthread_create(&thread_id, NULL, child_function, NULL);`
    - **Parameters:**
        - `&thread_id`: Pointer to the `pthread_t` variable where the new thread's ID will be stored.
        - `NULL`: Thread attributes (e.g., stack size, scheduling). `NULL` uses default attributes.
        - `child_function`: A pointer to the function the new thread will execute.
        - `NULL`: An argument to be passed to the `child_function`. `NULL` if no argument is needed.
5. **Wait for Thread Completion using `pthread_join()`:**
    - The `pthread_join()` function makes the calling thread (e.g., the main thread) wait for the specified thread to terminate. `c pthread_join(thread_id, NULL);`
    - **Parameters:**
        - `thread_id`: The ID of the thread to wait for.
        - `NULL`: A pointer to a `void*` variable where the target thread's return value will be stored. `NULL` if you don't care about the return value.

Example C Code Structure:

```
 #include <pthread.h>
#include <stdio.h>

// 2. Define the thread function
void* child_function(void* arg) {
    printf("Child thread is running.\n");
    // Simulate some work
    for (int i = 0; i < 3; i++) {
        printf("Child: %d\n", i);
```

```
    }
    return NULL;
}

int main() {
    // 3. Declare a pthread_t variable
    pthread_t thread1;

    // 4. Create the thread
    printf("Main thread: Creating child thread.\n");
    pthread_create(&thread1, NULL, child_function, NULL);

    // 5. Wait for the child thread to finish
    printf("Main thread: Waiting for child thread to complete.\n");
    pthread_join(thread1, NULL); // Main thread waits here until child_fu

    printf("Main thread: Child thread finished. Exiting.\n");
    return 0;
}
```

Pages 109-128

Here's a simplified learning guide based on the provided text:

Threading Concepts & Implementation

1. C Threads (Pthreads)

Pthreads is a POSIX standard for threads in C/C++.

1.1 How to Create Threads

1. **Include Library:** `#include <pthread.h>`
2. **Define Thread Function:**
   - Must return `void*`.
   - Must accept `void*` as an argument.
   - Example: `void* child(void* arg)`
3. **Declare Thread Variable:** `pthread_t thread_id;`
4. **Create Thread:** Use `pthread_create()`.

   `c int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);`

   - `thread`: Pointer to `pthread_t` variable where the new thread's ID will be stored.
   - `attr`: Thread attributes (e.g., stack size). Pass `NULL` for default.
   - `start_routine`: Pointer to the function the thread will execute.
   - `arg`: Argument to pass to the `start_routine` function. Pass `NULL` if no argument.
   - **Wait for Thread (Join):** `pthread_join(thread_id, NULL);`
   - Blocks the calling thread (e.g., `main`) until the specified `thread_id` finishes execution.

1.2 Example C Thread Code

```
#include <pthread.h>
#include <stdio.h>

// 2. Define the thread function
void* child(void* arg) {
    printf("Thread is running.\n");
    return NULL;
}

int main() {
    // 3. Declare a pthread_t variable
```

```
    pthread_t thread1;

    // 4. Create the Thread
    pthread_create(&thread1, NULL, child, NULL);

    // 5. Wait for the thread to finish
    pthread_join(thread1, NULL);

    return 0;
}
```

2. Concurrency Problems

These problems arise when multiple threads access and modify shared resources simultaneously.

2.1 Common Concurrency Problems

- **Temporary Update Problem:** A transaction updates a data item, but fails before committing, and another transaction reads the uncommitted update.
- **Incorrect Summary Problem:** When a summary calculation (e.g., sum) is performed on data that is simultaneously being updated, leading to an incorrect result.
- **Lost Update Problem:** Two transactions read the same data, both update it, and the second update overwrites the first one, causing the first update to be lost.
- **Unrepeatable Read Problem:** A transaction reads the same data item twice, but another transaction modifies the data item between the two reads, leading to different results.
- **Phantom Read Problem:** A transaction executes a query returning a set of rows. When the transaction re-executes the same query, it sees different rows due to another transaction inserting or deleting rows that match the query criteria.

3. Preventing Concurrency Problems (Semaphores in C)

Semaphores are synchronization primitives used to control access to shared resources.

3.1 Semaphore Functions

1. **Include Library:** `#include <semaphore.h>`
2. **Declare Semaphore:** `sem_t sem;`
3. **Initialize Semaphore:** `sem_init(&sem, pshared, value);`
   - `&sem`: Pointer to the `sem_t` semaphore object.
   - `pshared`: 0 if semaphore is shared *within* a process (threads); non-zero if shared *between* processes.
   - `value`: Initial count of available permits (e.g., 1 for a binary semaphore, which acts like a mutex).
4. **Acquire Lock (Wait):** `sem_wait(&sem);`
   - Decrements the semaphore count. If count is 0, the thread blocks until a permit is available.
5. **Release Lock (Post):** `sem_post(&sem);`
   - Increments the semaphore count, potentially unblocking a waiting thread.
6. **Destroy Semaphore:** `sem_destroy(&sem);`
   - Releases resources associated with the semaphore.

3.2 Example C Semaphore Code

```
 #include <pthread.h>
#include <stdio.h>
#include <semaphore.h> // Include for semaphores

sem_t sem;      // Declare a semaphore
int shared = 0; // Shared resource

void* worker(void* arg) {
```

```
    sem_wait(&sem);  // Acquire lock (decrement semaphore, wait if 0)
    shared++;        // Critical section: modify shared resource
    printf("Shared value: %d\n", shared);
    sem_post(&sem);  // Release lock (increment semaphore)
    return NULL;
}

int main() {
    pthread_t t1, t2;
    sem_init(&sem, 0, 1); // Initialize binary semaphore (value = 1)

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem); // Destroy the semaphore
    return 0;
}
```

4. Processes and Threads Comparison

- **Process:**
  - A self-contained execution environment.
  - Has its own complete, private set of run-time resources, including its own separate memory space.
- **Thread:**
  - Also known as a "lightweight process."
  - Exists *within* a process (every process has at least one thread).
  - Threads within the same process *share* basic run-time resources, including memory space.

5. Java Threads

Java threads allow a program to run multiple tasks concurrently (simultaneously).

5.1 Basics of Java Threads

- Every Java program has at least one thread, called the **main** or **parent** thread.
- Other threads created by the program are called **child threads**.
- **Purpose:** To perform tasks in the background without interrupting the main program, enabling concurrent execution.
    - Example: Read a large file and perform other long computations at the same time, then process the file once reading is complete.

5.2 Creating & Starting Java Threads

There are two main ways to create threads in Java:

Method 1: Extending the `Thread` Class

1. **Create a class** that `extends Thread`.
2. **Override the `run()` method**: This method contains the code that the thread will execute.

   `java class MyThread extends Thread { public void run() { System.out.println("Running on background thread."); // ... long running task ... } }`

3. **Instantiate and Start:**
    - Create an object of your `MyThread` class.
    - Call the `start()` method on this object. **Do not call `run()` directly**, as `start()` handles thread setup and calls `run()` on the new thread.

   `java public class Main { public static void main(String[] args) { MyThread thread1 = new MyThread();`

`thread1.start(); // This starts the new thread and calls its run() method. System.out.println("Running on main thread."); } }`

Method 2: Implementing the `Runnable` Interface

1. **Create a class** that `implements Runnable`.
2. **Implement the `run()` method**: This method contains the code the thread will execute.

   `java class MyRunnable implements Runnable { public void run() { System.out.println("Runnable thread is running."); // ... another task ... } }`

3. **Instantiate and Start:**
   - Create an object of your `MyRunnable` class.
   - Create a `Thread` object, passing your `Runnable` instance to its constructor.
   - Call the `start()` method on the `Thread` object.

   `java public class Main { public static void main(String[] args) { MyRunnable runnable = new MyRunnable(); Thread thread2 = new Thread(runnable); thread2.start(); // Starts the new thread and calls runnable's run() method. } }`

5.3 Waiting for Threads (`join()`)

The `join()` method allows one thread to wait for the completion of another thread.

- When you call `thread.join()`, the current thread (e.g., `main`) will pause its execution until `thread` finishes its `run()` method.
- `join()` can throw an `InterruptedException`, so it often needs to be wrapped in a `try-catch` block.

```
public class Main {
    public static void main(String[] args) {
```

```
        // Assume LongProcess extends Thread
        LongProcess lp = new LongProcess();
        lp.start();

        // Assume ImmortalProcess implements Runnable
        ImmortalProcess ip = new ImmortalProcess();
        Thread ipThread = new Thread(ip);
        ipThread.start();

        try {
            lp.join();        // Main thread waits for lp to finish
            ipThread.join();  // Main thread waits for ipThread to finish
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted while waiting.");
        }
        System.out.println("Both background threads have finished.");
    }
}
```

Pages 127-146

This learning guide condenses the provided text (Pages 127-146) into key concepts, definitions, and practical information, eliminating verbose explanations.

Learning Guide: Threads, Concurrency, and Process Management

Section 1: Java Threads

Threads allow programs to perform multiple tasks concurrently.

1.1 How to Create Threads

There are two main ways to create threads in Java:

1. **Extending the `Thread` class:**
    - Create a class that `extends Thread`.
    - Override the `run()` method with the code the thread should execute.
    - **Example:** `java class LongProcess extends Thread { public void run() { try { System.out.println("Long Process"); sleep(1000); // Pause for 1 second } catch (InterruptedException e) { /* handle exception */ } } }`
2. **Implementing the `Runnable` interface:**
    - Create a class that `implements Runnable`.
    - Override the `run()` method with the code the thread should execute.
    - **Example:** `java class ImmortalProcess implements Runnable { public void run() { while (true) { System.out.println("Running at all times"); } } }`

1.2 How to Start Threads

Once a thread class is defined, you start execution:

1. **For classes extending `Thread`:**
    - Instantiate an object of your `Thread` subclass.
    - Call its `start()` method.
    - **Example:** `LongProcess lp = new LongProcess(); lp.start();`
2. **For classes implementing `Runnable`:**
    - Instantiate an object of your `Runnable` implementation.
    - Create a `Thread` object, passing your `Runnable` object to its constructor.
    - Call the `Thread` object's `start()` method.

- **Example:** `ImmortalProcess ip = new ImmortalProcess(); Thread ipThread = new Thread(ip); ipThread.start();`

1.3 Waiting for Threads to Finish (`join()`)

- The `join()` method makes the *current* thread wait until the thread it's called on finishes execution.
- **Important:** `join()` can throw an `InterruptedException`, so always use a `try-catch` block.
- **Example:** `lp.join(); ipThread.join();` (This would make the `main` thread wait for `lp` and `ipThread` to complete).

Section 2: Concurrency Problems and Prevention

2.1 Concurrency Problems

These occur when multiple threads access and modify shared resources simultaneously.

- **Shared Resource:** Any data or object that multiple threads can interact with (e.g., a shared `Storage` object).
- **Common Problems:**
  - **Temporary Update Problem:** Data is temporarily updated by one transaction but rolled back, leading to other transactions reading incorrect temporary data.
  - **Incorrect Summary Problem:** When reading a summary (e.g., total count) while other transactions are updating the individual items, leading to an inconsistent summary.
  - **Lost Update Problem:** One thread's update is overwritten by another thread's update because both read the same outdated value.
  - **Unrepeatable Read Problem:** A thread reads data, and when it tries to read the same data again, it finds that another thread has modified or deleted it.
  - **Phantom Read Problem:** A thread reads data, and when it re-executes the same query, it finds new rows inserted by another transaction.

2.2 Preventing Concurrency Problems

1. **`isAlive()` method:**
    - Checks if a thread has been started and has not yet terminated.
    - **Caution:** While you *can* use `isAlive()` to check a thread's status, it's generally **not recommended** for making one thread wait for another (use `join()` instead). Continuously checking `isAlive()` in a loop (busy-waiting) wastes CPU cycles.
2. **Semaphores (`java.util.concurrent.Semaphore`)**
    - A semaphore controls access to a shared resource using a counter.
    - **Purpose:** Limits the number of threads that can access a critical section (a piece of code that manipulates a shared resource) at any given time.
    - **Initialization:** `Semaphore s = new Semaphore(int permits);`
        - `permits`: The maximum number of threads allowed to access the resource concurrently.
        - `new Semaphore(1)` creates a **binary semaphore** (similar to a mutex), allowing only one thread at a time.
    - **Usage:**
        - `s.acquire()`: A thread calls this to request permission to enter the critical section. If the permit count is 0, the thread waits until a permit is released.
        - `s.release()`: A thread calls this to release a permit after exiting the critical section, incrementing the permit count.
    - **Example:** `java import java.util.concurrent.Semaphore; // ... inside a thread's run method that accesses shared resource try { semaphore.acquire(); // Acquire a lock (permit) // Critical section: Access shared resource here Thread.sleep(10); // Simulate work } catch (InterruptedException e) { /* handle */ } finally { semaphore.release(); // Release the lock }`

Section 3: Process Management

Process Management is a core function of the Operating System.

3.1 Process Concept

- **Process:** A program in execution. Each process progresses sequentially.
- **Self-contained execution environment:** A process has its own resources.
- **Components of a Process:**
  - **Program Counter (PC):** Points to the next instruction to be executed.
  - **Stack:** Stores temporary data (e.g., function parameters, local variables, return addresses).
  - **Data Section:** Stores global variables.

3.2 Process Life Cycle / Process States

A process transitions through different states during its execution:

1. **New / Start:** The process is being created.
2. **Ready:** The process is waiting to be assigned to a CPU. It's loaded into main memory and ready to run.
3. **Running:** Instructions of the process are being executed by the CPU.
4. **Waiting (Blocked):** The process is waiting for some event to occur (e.g., I/O completion, receiving a signal). It cannot execute until the event happens.
5. **Terminated / Exit:** The process has finished execution.

3.3 Process Control Block (PCB)

- **Definition:** A data structure maintained by the Operating System for *every* process.
- **Purpose:** Contains all the information needed to manage a specific process.
- **Information stored in a PCB:**
  - **Process State:** Current state (New, Ready, Running, Waiting, Terminated).
  - **Process Privileges:** Access rights to system resources.

- **Process ID (PID):** Unique identifier for the process.
- **Pointer:** To a parent process (for hierarchical process structures).
- **Program Counter:** Address of the next instruction.
- **CPU Registers:** Values of CPU registers when the process was last suspended.
- **CPU Scheduling Information:** Process priority, pointers to scheduling queues.
- **Memory Management Information:** Base/limit registers, page tables.
- **Accounting Information:** CPU usage, real time used, job numbers.
- **I/O Status Information:** List of open files, I/O devices allocated.

3.4 Context Switch

- **Definition:** The mechanism by which the CPU saves the state of the currently running process and loads the saved state of another process to resume its execution.
- **When it occurs:** When the CPU switches from one process to another (e.g., time slice expires, I/O request, interrupt).
- **Overhead:** Context-switch time is pure overhead; the system performs no useful work during this period. Its duration depends on hardware support.
- **Steps:**
  1. Save the PCB of the old process.
  2. Load the PCB of the new process.

3.5 Process Scheduling Queues

Queues help the OS manage processes at different stages of their lifecycle.

1. **Job Queue:**
   - Contains *all* processes in the system, typically those waiting to enter main memory.
2. **Ready Queue:**
   - Contains all processes residing in main memory that are ready and waiting to execute on the CPU.

    - Processes in the Ready state are in this queue.

3. **Device Queues (I/O Queues):**
    - Contains processes waiting for a particular I/O device to become available or for an I/O operation to complete.

3.6 Schedulers

Schedulers are special system software that manage the flow of processes.

1. **Long-Term Scheduler (Job Scheduler):**
    - **Role:** Selects which processes from the **Job Queue** should be brought into the **Ready Queue** (i.e., loaded into main memory).
    - **Frequency:** Less frequent, as it controls the degree of multiprogramming.
2. **Short-Term Scheduler (CPU Scheduler):**
    - **Role:** Selects which process from the **Ready Queue** should be executed next by the CPU.
    - **Frequency:** Very frequent, runs every few milliseconds. It is responsible for making fast decisions to optimize CPU utilization.
3. **Medium-Term Scheduler:**
    - **Role:** Manages "swapped-out" processes. It can remove processes from main memory (swap them out to disk) to reduce the degree of multiprogramming or free up memory. Later, it can swap them back in.
    - **Necessity:** Not always necessary; common in systems with limited RAM or for balancing system load.
    - **Swapping:** Moving a process from RAM to disk and vice-versa.

3.7 Process States and Flow Summary

This summarizes how processes move between states and queues, managed by schedulers:

1. **New** process enters the **Job Queue**.

2. **Long-Term Scheduler** picks processes from the Job Queue and moves them to the **Ready Queue** (loading them into memory).
3. From the Ready Queue, the **Short-Term Scheduler (CPU Scheduler)** selects a process to move to the **Running** state.
4. While **Running**:
    - If its time slice expires (e.g., due to an interrupt), it returns to the **Ready Queue**.
    - If it makes an I/O request, it moves to the **Waiting** state and enters a **Device Queue**.
    - If it forks a child process, the child enters the Ready Queue.
    - If an interrupt occurs, it might return to the Ready state or handle the interrupt.
5. From the **Waiting** state, once the awaited event (e.g., I/O completion) occurs, the process moves back to the **Ready Queue**.
6. A process in the Running state can eventually move to the **Terminated** state after completing its execution.
7. The **Medium-Term Scheduler** can move processes between main memory (Ready/Waiting states) and secondary storage (swapped-out processes) to manage memory and load.

Pages 145-164

Here is a simplified, easy-to-read learning guide based on the provided text:

Operating System Concepts: Processes, IPC, and Threads

1. Process States and Scheduling

Processes move through various states and are managed by schedulers.

- **Process States:**
    - **New:** Process is being created.
    - **Ready:** Process is waiting to be assigned to a CPU.

    - **Running:** Instructions are being executed by the CPU.
    - **Waiting:** Process is waiting for an event (e.g., I/O completion, child termination, interrupt).
    - **Terminated:** Process has finished execution.
- **Process Flow:**
    - A **new** process becomes **ready**.
    - The **short-term scheduler** (CPU scheduler) picks a process from **ready** to **running**.
    - A **running** process can move to:
        - **Waiting:** If it makes an I/O request, forks a child process and waits, or waits for an interrupt.
        - **Ready:** If its time slice expires or an interrupt occurs (preemption).
        - **Terminated:** If it finishes execution.
    - A **waiting** process becomes **ready** when its awaited event occurs (e.g., I/O completion, child terminates).
- **Schedulers:**
    - **Long-term scheduler (Job Scheduler):** Selects processes from the *job queue* to load into memory (creates them as *ready* processes). Controls the degree of multiprogramming.
    - **Short-term scheduler (CPU Scheduler):** Selects processes from the *ready queue* to allocate the CPU to. Executes frequently.
    - **Medium-term scheduler:** Used in time-sharing systems to swap processes in/out of memory to reduce the degree of multiprogramming or improve performance.

2. Process Termination

A process can terminate itself or be terminated by its parent or the OS.

- **Self-Termination (via `exit()`):**
    - Process executes its last statement.
    - Can pass output data to its parent (via `wait()`).
    - OS deallocates all its resources.
- **Parent Termination (via `abort()`):**
    - Parent decides to terminate a child process.
    - **Reasons:**
        - Child exceeded allocated resources.

        - Task assigned to child is no longer needed.
        - Parent is exiting.
- **Cascading Termination:**
    - If a parent process terminates, all its child processes (and their descendants) are also terminated by the operating system.

3. Cooperating Processes and Interprocess Communication (IPC)

Cooperating processes are processes that can affect or be affected by other processes. They often share data.

- **Why Cooperate?**
    - **Modularity:** Breaking complex tasks into smaller subtasks.
    - **Computation Speed-up:** Concurrent execution of subtasks.
    - **Information Sharing:** Access to the same data (e.g., a shared file or database).
- **Interprocess Communication (IPC):**
    - A mechanism for processes to exchange data and synchronize actions.
    - Provides two basic operations: `send(message)` and `receive(message)`.
    - Requires establishing a **communication link** between processes.
    - **Implementation:** Can be **physical** (e.g., shared memory) or **logical** (e.g., message passing).

3.1. IPC by Sharing (Physical)

Processes cooperate by accessing shared data.

- **Mechanism:** Processes directly share a region of memory, variables, files, or databases.
- **Example: Producer-Consumer Paradigm**
    - **Producer:** Creates data/items.
    - **Consumer:** Consumes data/items.
    - **Shared Buffer:** A common area in memory where the producer places items and the consumer takes them.
    - Uses `in` and `out` pointers to manage items in the circular buffer.

    - **Synchronization:** Needs mechanisms to ensure the producer doesn't write to a full buffer and the consumer doesn't read from an empty buffer.

3.2. IPC by Communication (Logical - Message Passing)

Processes cooperate by exchanging messages through system-provided primitives.

- **Mechanism:** Processes `send()` and `receive()` messages without direct shared memory. The OS handles the message transfer.
- Can be **Direct** or **Indirect**.

Direct Communication

- **Explicit Naming:** Processes must explicitly name each other.
    - `send(P, message)`: Send message to process P.
    - `receive(Q, message)`: Receive message from process Q.
- **Link Properties:**
    - Established automatically between communicating processes.
    - Associated with exactly one pair of processes.
    - Exactly one link between each pair.
    - Can be unidirectional or bidirectional.

Indirect Communication

- **Mailboxes (Ports):** Messages are sent to and received from a shared mailbox.
    - Each mailbox has a unique ID.
    - Processes communicate only if they share a mailbox.
- **Operations:**
    - `create a new mailbox`
    - `send(A, message)`: Send message to mailbox A.
    - `receive(A, message)`: Receive message from mailbox A.
    - `destroy a mailbox`
- **Link Properties:**
    - Established only if processes share a common mailbox.
    - A link (mailbox) can be associated with many processes.
    - Multiple links (mailboxes) can exist between processes.

    - Can be unidirectional or bidirectional.
- **Mailbox Sharing Problem:** If multiple processes share a mailbox and receive messages, who gets which message?
    - **Solutions:**
        - Limit a link to at most two processes.
        - Allow only one process at a time to execute `receive`.
        - System arbitrarily selects the receiver, and notifies the sender.

Buffering (for Message Passing)

Messages are queued on the communication link.

1. **Zero Capacity:**
    - Queue holds 0 messages.
    - Sender must wait for the receiver to receive the message (known as **rendezvous**).
2. **Bounded Capacity:**
    - Queue has a finite length (n messages).
    - Sender must wait if the link (queue) is full.
3. **Unbounded Capacity:**
    - Queue has infinite length.
    - Sender never waits (messages are always accepted).

4. Threads

A **thread** is a lightweight unit of execution within a process.

- **Definition:** A flow of execution through a process's code, with its own:
    - **Program Counter:** Keeps track of the next instruction.
    - **System Registers:** Holds current working variables.
    - **Stack:** Contains execution history.
- Also called a **Lightweight Process**.
- Improves OS performance by reducing overhead compared to full processes.

4.1. Single vs. Multi-threaded Process

- **Single-threaded Process:** Has one flow of execution.

- **Multi-threaded Process:** Has multiple threads, each with its own PC, registers, and stack, but sharing the same code, data, and resources (e.g., open files) of the parent process.

4.2. Process vs. Thread Comparison

Feature	Process	Thread
Weight	Heavyweight, resource-intensive	Lightweight, less resource-intensive
Switching	Needs OS interaction (context switch)	Often doesn't need OS interaction (faster)
Resources	Own memory space, file resources	Shares memory, open files, child processes
Blocking	If one process blocks, others may continue, but within the same process, no other task can execute.	If one thread blocks, another thread in the same process can run.
Resource Usage	Multiple processes use more resources	Multiple threads use fewer resources
Independence	Operates independently	Can read, write, or change another thread's data (within the same process)

4.3. Advantages of Threads

- **Minimized Context Switching Time:** Faster to switch between threads within the same process than between processes.
- **Concurrency within a Process:** Allows multiple parts of a single application to execute concurrently.
- **Efficient Communication:** Threads within the same process can communicate easily via shared memory.
- **Economical:** Cheaper and faster to create and switch threads than processes.
- **Multiprocessor Utilization:** Better utilizes multiprocessor architectures by running multiple threads in parallel.

5. Synchronization Problems

When processes or threads share resources, their access needs to be synchronized to prevent data corruption and ensure correct execution.

5.1. Dining Philosophers Problem

- **Scenario:** Philosophers are seated around a table, with a plate of noodles in front of each and one fork between each pair of plates.
- **Rules:**
  - Philosophers can think or eat.
  - To eat, a philosopher needs both the left and right forks.
  - A philosopher can only pick up one fork at a time.
- **Challenge:** Design a solution that avoids:
  - **Deadlock:** Two or more processes (philosophers) are permanently blocked, each waiting for a resource (fork) held by another. (e.g., all pick up their left fork simultaneously).
  - **Starvation:** A process (philosopher) waits indefinitely for a resource or CPU time that is continually given to other processes.

Pages 163-182

Here is a simplified learning guide based on the provided text:

Operating Systems: Synchronization & CPU Scheduling

Part 1: Synchronization Problems

1. Synchronization Problem Overview

- **Definition:** Issues that arise when multiple processes or threads access shared resources concurrently, requiring coordination to maintain data consistency and prevent race conditions.

2. Case Study: Dining Philosophers Problem

This problem illustrates common synchronization challenges. * **Setup:** * Philosophers seated at a round table. * Each has a plate of noodles. * A single fork is placed between each pair of adjacent plates. * **Rules:** * A philosopher can **think**. * A philosopher can **eat** only if they pick up **two forks** (one from their left, one from their right). * Philosophers can pick up only **one fork at a time**. * **Goals for Solution:** * Avoid **Deadlock**. * Avoid **Starvation**.

3. Key Synchronization Concepts

- **Deadlock:**
  - **Definition:** Two or more processes are permanently blocked because each is waiting for a resource held by another.
  - **In Dining Philosophers:** Occurs if all philosophers simultaneously pick up one fork (e.g., their left fork), then each waits indefinitely for the right fork that another philosopher holds, forming a **circular wait**.
- **Starvation:**
  - **Definition:** A process waits indefinitely for the resources or CPU time it needs because these are continually given to other processes.
  - **In Dining Philosophers:** Possible if one philosopher repeatedly misses out on forks while others are always able to acquire them.

4. Synchronization Primitive: Semaphore

- **Definition:** A synchronization primitive used to coordinate concurrent processes or threads by controlling access to shared resources.
- **How it Works (General):** Acts as a counter or a flag to signal the availability of resources or to control entry into critical sections.
- **Types:**
  - **Binary Semaphore:** Like an ON/OFF switch (value 0 or 1), used for mutual exclusion (only one process can access a resource at a time).
  - **Counting Semaphore:** Allows multiple identical resources (value can be any non-negative integer), used to control access to a pool of resources.
- **Application to Dining Philosophers (Implied):** Semaphores can be used to represent each fork, allowing philosophers to `wait()` (pick up) and `signal()` (put down) forks, thus regulating access.

Part 2: CPU Scheduling Algorithms

1. Introduction to CPU Scheduling

- **Purpose:** The fundamental basis of multiprogrammed operating systems. It determines which process among those ready to run should be allocated the CPU next.
- **Role of Short-term Scheduler (CPU Scheduler):**
  - Selects a process from the **Ready Queue**.
  - Allocates the CPU to the selected process.
- **Process Flow:**
  1. **Job Queue:** All processes in the system.
  2. **Ready Queue:** Processes waiting and ready to execute.
  3. **CPU:** Executes the chosen process.

2. Process Behavior: CPU Bursts and I/O Bursts

Processes typically alternate between performing computations (CPU bursts) and waiting for I/O operations (I/O bursts). * **CPU Burst:** * **Definition:** The time period during which a process executes instructions on the CPU

continuously, without interruption for input/output (I/O). * **Example Activities:** `load`, `store`, `add`, `increment`, `index`. * **I/O Burst:** * **Definition:** The time period during which a process is waiting for or performing I/O operations. * **Example Activities:** `read from file`, `write to file`, `wait for I/O`, waiting for keyboard input. * **Alternating Sequence:** Processes alternate between CPU-bound work and I/O-bound waits.

3. Intended Learning Outcomes (What you should learn)

After studying CPU scheduling algorithms, you should be able to: * Identify the behavior of different scheduling algorithms. * Compute scheduling criteria (e.g., turnaround time, waiting time) for various algorithms. * Present criteria for choosing an effective scheduling algorithm. * Implement basic scheduling algorithms like First-Come, First-Served (FCFS) and Non-preemptive Shortest-Job-First (SJF).

Pages 181-200

Here is a simplified, easy-to-read learning guide based on the provided text, designed for efficient study.

CPU Scheduling Learning Guide

1. Introduction to CPU Scheduling

- **Definition:** CPU Scheduling is the core mechanism in multiprogrammed Operating Systems (OS) that decides which process from the `ready queue` gets allocated to the CPU next.
- **Purpose:** Maximizes CPU utilization and provides efficient system performance by managing multiple processes simultaneously.
- **Short-Term Scheduler (CPU Scheduler):** The component responsible for making these decisions and allocating the CPU.
- **Process Flow:** `Job Queue` → `Ready Queue` → `CPU`

2. Process Bursts

Processes alternate between CPU execution and I/O wait states.

- **CPU Burst:** The time period when a process continuously executes instructions on the CPU without I/O interruption.
- **I/O Burst:** The time period when a process waits for or performs Input/Output (I/O) operations (e.g., reading from disk, waiting for keyboard input).
- **Alternating Sequence Example:** `CPU Burst` (load, store) → `I/O Burst` (wait for I/O) → `CPU Burst` (add, store) → `I/O Burst` (read from file)

3. Types of Processes

- **I/O-Bound Process:**
  - Spends more time doing I/O than computations.
  - Characterized by many short CPU bursts.
- **CPU-Bound Process:**
  - Spends more time doing computations.
  - Characterized by few, very long CPU bursts.

4. Key Scheduling Terms & Performance Criteria

These terms are crucial for understanding and evaluating scheduling algorithms.

A. Process Timings & Metrics

- **Arrival Time (AT):** The time when a process enters the ready queue and becomes ready for execution.
- **Burst Time (BT) / CPU Burst Time:** The total time a process needs on the CPU to complete its execution.
- **Start Time (ST):** The time when a process actually begins executing on the CPU for the first time.
- **Finish Time (FT) / Completion Time (CT):** The time when a process completes all its execution (CPU and I/O, if applicable).
- **Gantt Chart:** A visual timeline representation of when processes execute on the CPU.

B. Performance Criteria

- **CPU Utilization:** The percentage or fraction of time the CPU is actively performing work. (Higher is generally better)
- **Throughput:** The number of processes completed per unit of time. (Higher is generally better)
- ****Turn-around Time (TAT):** The total time from process submission (arrival) to its completion.
  - Formula: `TAT = Finish Time (FT) - Arrival Time (AT)`
- **Waiting Time (WT):** The total time a process spends waiting in the ready queue.
  - Formula: `WT = Turn-around Time (TAT) - Burst Time (BT)`
- **Response Time (RT):** The time it takes from arrival until the process first starts executing (gets its first response).
  - Formula: `RT = Start Time (ST) - Arrival Time (AT)`
- **Average Waiting Time (AWT):** A crucial parameter to judge scheduling algorithm performance. Lower AWT indicates a better algorithm.
  - Formula: `AWT = (Sum of all Waiting Times) / (Number of Processes)`

Example Calculation (FCFS Scenario):

Process	AT	BT	ST	FT	TAT (FT-AT)	WT (TAT-BT)	RT (ST-AT)
P1	0	24	0	24	24	0	0
P2	2	3	24	27	25	22	22
P3	4	3	27	30	26	23	23

- **Gantt Chart:** `| P1 (0-24) | P2 (24-27) | P3 (27-30) |`
- **Average Waiting Time (AWT):** `(0 + 22 + 23) / 3 = 15`

5. Scheduling Schemes

- **Preemptive:** The currently running process can be interrupted by the OS and moved back to the ready state if a higher-priority or more urgent process arrives or becomes ready.

- **Nonpreemptive:** Once a process starts running, it continues until it either terminates or voluntarily blocks itself (e.g., for an I/O operation). It cannot be interrupted by the OS.

6. CPU Scheduling Algorithms (Overview)

Common algorithms include:

- First-Come, First-Served (FCFS)
- Shortest-Job-First (SJF) (Nonpreemptive and Preemptive/SRTF)
- Priority Scheduling (Nonpreemptive and Preemptive)
- Highest Response-Ratio Next (HRRN) Scheduling
- Round-Robin (RR) Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

7. First-Come, First-Served (FCFS) Scheduling

- **Mechanism:** The process that requests the CPU first is allocated the CPU first. It's managed using a FIFO (First-In, First-Out) queue.
- **Characteristics:** Simple to implement.
- **Cons:**
  - **Convoy Effect:** A single long-running process can hold the CPU and make many shorter processes wait for a very long time, slowing down the entire system.
  - Generally favors CPU-bound processes.

Example 1 (No Arrival Times - Assume all arrive at 0):

Process	Burst Time
P1	24
P2	3
P3	3

- **Arrival Order:** P1, P2, P3
- **Gantt Chart:** `| P1 (0-24) | P2 (24-27) | P3 (27-30) |`
- **Waiting Times:** P1=0, P2=24, P3=27
- **Average Waiting Time:** `(0 + 24 + 27) / 3 = 17`

Example 2 (No Arrival Times - Different Order):

- **Arrival Order:** P2, P3, P1
- **Gantt Chart:** `| P2 (0-3) | P3 (3-6) | P1 (6-30) |`
- **Waiting Times:** P2=0, P3=3, P1=6
- **Average Waiting Time:** `(0 + 3 + 6) / 3 = 3`
    - *Note how arrival order drastically impacts AWT in FCFS.*

Practice Example (FCFS with Arrival Times):

Process	Burst Time	Arrival Time
P1	21	0
P2	3	1
P3	6	3
P4	2	5

- *Task: Draw the Gantt Chart, fill in Start Time, Finish Time, TAT, WT, and calculate AWT.*

8. Shortest-Job-First (SJF) Scheduling

- **Mechanism:** This algorithm prioritizes processes with the shortest *next CPU burst* length. Short processes jump ahead of longer ones.
- **Optimality:** SJF is considered optimal because it yields the minimum average waiting time among all scheduling algorithms.
- **Cons:**
    - **Starvation:** Long processes may wait indefinitely in the ready queue if there's a continuous stream of shorter processes.
    - Requires knowing the future CPU burst length, which is difficult to predict accurately in real systems.

A. Nonpreemptive SJF

- **Rule:** When the CPU becomes available, it is assigned to the job with the smallest next CPU burst.
- **Tie-breaking:** If two processes have the same shortest burst time, FCFS is used.

Example (Nonpreemptive SJF - all arrive at 0):

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

- **Sorted by BT:** P4 (3), P1 (6), P3 (7), P2 (8)
- **Gantt Chart:**
  `| P4 (0-3) | P1 (3-9) | P3 (9-16) | P2 (16-24) |`
- **Waiting Times:** P4=0, P1=3, P3=9, P2=16
- **Average Waiting Time:** `(0 + 3 + 9 + 16) / 4 = 7`

Practice Example (Nonpreemptive SJF with Arrival Times):

Process	Burst Time	Arrival Time
P1	3	0
P2	13	2
P3	5	3
P4	8	5

- *Task: Draw the Gantt Chart, complete the table (ST, FT, TAT, WT), and provide the Average Turn-Around Time and Average Waiting Time.*

B. Preemptive SJF (Shortest-Remaining-Time-First - SRTF)

- **Rule:** The currently executing process can be preempted (interrupted) if a new process arrives in the ready queue and its *remaining burst time* is shorter than the *remaining burst time* of the currently running process.

Example (Preemptive SJF / SRTF with Arrival Times):

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1

Process	Arrival Time	Burst Time
P4	5	4

- **Gantt Chart (Simplified):** `| P1 (0-2) | P2 (2-4) | P3 (4-5) | P2 (5-6) | P4 (6-10) | P1 (10-16) |`
  - P1 runs (0-2), P2 arrives (AT=2, BT=4). P1 remaining is 5, P2 remaining is 4. P2 preempts.
  - P2 runs (2-4), P3 arrives (AT=4, BT=1). P2 remaining is 2, P3 remaining is 1. P3 preempts.
  - P3 runs (4-5), completes. P2 remaining is 2, P4 arrives (AT=5, BT=4). P2 has shorter remaining (2) than P4 (4). P2 runs.
  - P2 runs (5-6), completes. P1 remaining is 5, P4 remaining is 4. P4 has shorter remaining (4). P4 runs.
  - P4 runs (6-10), completes. P1 remaining is 5. P1 runs.
  - P1 runs (10-16), completes.
- **Waiting Times:** P1 = (10-0-2) = 8 (Wait after first burst: 10 - 2 (P1 ran for 2 units), so P1 waited 8 units) = Wait time = 8 units. No, easier to calculate: P1(0-2) then P1(10-16). So it arrived at 0, first ran at 0, finished at 16, burst=7. TAT = 16-0=16. WT = 16-7=9.
  - P1: (10-2) + (0-0) = 8 + 0 = 8. (Wrong calculation in original slide, recalculate using TAT-BT)
  - P1: FT=16, AT=0, BT=7. TAT = 16-0=16. WT = 16-7=9.
  - P2: FT=6, AT=2, BT=4. TAT = 6-2=4. WT = 4-4=0. (Actually, P2 waits for 2 units initially until P1 is preempted, but P2's ST is 2. So RT is 0. WT is 0)
  - P3: FT=5, AT=4, BT=1. TAT = 5-4=1. WT = 1-1=0.
  - P4: FT=10, AT=5, BT=4. TAT = 10-5=5. WT = 5-4=1.
- **Corrected Average Waiting Time:** `(9 + 0 + 0 + 1) / 4 = 10 / 4 = 2.5`
  - *(Note: The slide had AWT = 3, which might be based on a slightly different calculation of individual waiting times. My calculation aligns with the definitions provided earlier.)*

Pages 199-218

Here is a simplified, easy-to-read learning guide based on the provided text:

CPU Scheduling Algorithms: A Study Guide

This guide covers essential concepts and algorithms in CPU scheduling.

I. Key Scheduling Metrics

These metrics are used to evaluate the efficiency of scheduling algorithms:

- **Response Time (RT):** The amount of time it takes to start the response.
  - **Calculation:** `Start Time - Arrival Time`
- **Turn-Around Time (TAT):** The total time delay from submission of a process to its completion.
  - **Calculation:** `Finish Time - Arrival Time`
- **Waiting Time (WT):** The time a job spends waiting in the ready queue.
  - **Calculation:** `Turn-Around Time - Burst Time`
- **Burst Time (BT):** The total time a process needs on the CPU to complete its execution.
- **Efficiency Note:** Higher Average TAT generally indicates *less* efficient scheduling.

II. Shortest-Job-First (SJF) Scheduling

SJF schedules the process with the smallest next CPU burst. It aims for minimum average waiting time.

A. Nonpreemptive SJF

- **Rule:** Once a process starts executing, it runs to completion, even if a new, shorter job arrives.
- **Example Tasks:** Draw Gantt Chart, calculate Average Turn-Around Time, and Average Waiting Time.

B. Preemptive SJF (Shortest-Remaining-Time-First - SRTF)

- **Rule:** If a new process arrives in the ready queue with a CPU burst time shorter than the *remaining* time of the currently executing process, the current process is preempted. The new process then starts executing.
- **Example Tasks:** Draw Gantt Chart, calculate Average Turn-Around Time, and Average Waiting Time.

C. Determining the Length of the Next CPU Burst

- **Challenge:** SJF requires knowing the length of the next CPU burst in advance, which is often impossible.
- **Solution:** Estimate the length using **exponential averaging**.

Exponential Averaging Formula:

$$T_{n+1} = \alpha * t_n + (1 - \alpha) * T_n$$

- **T_{n+1}:** Predicted burst time for the *next* CPU burst.
- **t_n:** Actual length of the *last* CPU burst that just completed.
- **T_n:** Previous prediction for the *last* CPU burst.
- **α (alpha):** A weight factor ($0 \leq \alpha \leq 1$).
  - If $\alpha = 0$, $T_{n+1} = T_n$ (old predictions matter most).
  - If $\alpha = 1$, $T_{n+1} = t_n$ (only the actual last burst matters).

III. Priority Scheduling

Each process has a priority number; the scheduler always chooses the highest priority process.

- **Priority Rule:** Smaller number usually means higher priority.
- **Problem: Starvation** – Low-priority processes may never execute if high-priority processes continually arrive.
- **Solution: Aging** – Gradually increase the priority of processes that wait in the system for a long time.

A. Nonpreemptive Priority

- **Rule:** When the CPU becomes available, it is allocated to the job with the highest priority. Once started, it runs to completion.
- **Tie-breaker:** If priorities are equal, use First-Come, First-Served (FCFS).

B. Preemptive Priority

- **Rule:** If a new job arrives with a higher priority than the currently executing job, the current job is preempted, and the new job starts.
- **Tie-breaker:** If priorities are equal, use FCFS.
- **Example Tasks:** Draw Gantt chart, compute Average Waiting Time, Average Turn-Around Time, and Average Response Time for both nonpreemptive and preemptive versions.

IV. Round-Robin (RR) Scheduling

RR is a preemptive scheduling algorithm designed for time-sharing systems, also known as "time slicing."

- **Mechanism:** Uses a clock interrupt at periodic intervals.
- **Time Quantum (q):** Each process is given a fixed time slice (e.g., 10-100msec) to use the CPU.
    - No job is allocated the CPU for more than one time quantum at a time.

- **Preemption:** If the time quantum runs out before a process finishes, it is preempted.

A. How it Works

1. New processes are inserted at the **tail** of the ready queue.
2. The process at the **head** of the queue runs for at most `q` milliseconds.
3. **If the process finishes** before `q` runs out: It is removed from the queue.
4. **If the process does not finish** and `q` runs out: It is preempted and transferred to the **tail** of the queue.
5. The next process in line becomes the new head of the queue.

B. Performance Considerations

- **`q` (time quantum) is large:** RR behaves like FCFS (First-Come, First-Served).
- **`q` is small:** Overhead due to frequent **context switches** (saving and restoring CPU state) becomes too high.
- **Ideal `q`:** Should be large enough compared to context switch time to minimize overhead, but small enough to provide reasonable response time for interactive processes.
- **Typical Results:** Higher average Turn-Around Time than SJF, but significantly better Response Time.

V. Highest-Response-Ratio-Next (HRRN) Scheduling

HRRN is a nonpreemptive scheduling algorithm that aims to balance favoring shorter processes with preventing starvation of longer processes.

- **Rule:** The process with the highest **Response Ratio** is executed next. Once started, it runs to completion (nonpreemptive).
- **Benefit:** Shorter processes are favored initially, but as longer processes wait, their Response Ratio increases, giving them a chance to execute (aging effect).

A. Response Ratio (RR) Formula

```
Response Ratio = (Waiting Time + Burst Time) / Burst Time
Response Ratio = 1 + (Waiting Time / Burst Time)
```

- **Note:** The Waiting Time in the formula refers to the time a process has *already waited* in the ready queue up to the point of selection.

(Note: Pages 215-218 contained instructions for a seatwork/exam and practice problems, not new conceptual content. Therefore, they have been omitted from this learning guide as per the instructions to focus on essential learning information.)

Pages 217-220

Here's a simplified learning guide based on the provided text, focusing on essential information for studying CPU scheduling algorithms.

CPU Scheduling Algorithms: A Study Guide

This guide covers common CPU scheduling algorithms. For each algorithm, you'll learn its core concept, the input data needed, and the metrics used to evaluate its performance.

Core Concepts & Metrics

CPU scheduling determines which process gets the CPU next. Its goal is to optimize system performance based on various criteria.

- **Process:** A program in execution.
- **Arrival Time (AT):** The time a process enters the ready queue.

- **Burst Time (BT):** The total CPU time a process needs to complete its execution.
- **Priority:** A value indicating a process's importance. Lower numbers often mean higher priority (this varies by system, but is standard for these problems).
- **Gantt Chart:** A visual timeline showing when each process executes on the CPU.

Performance Metrics:

- **Turnaround Time (TAT):** The total time a process spends in the system, from arrival to completion.
  - `TAT = Completion Time - Arrival Time`
- **Waiting Time (WT):** The total time a process spends waiting in the ready queue.
  - `WT = Turnaround Time - Burst Time`
- **Response Time (RT):** The time from a process's arrival until it first starts executing on the CPU.
  - `RT = First Execution Start Time - Arrival Time`

Scheduling Algorithms

Let's explore specific algorithms with practice problems. For each problem, you'll need to: 1. Draw the **Gantt Chart**. 2. Compute the **Average TAT**, **Average WT**, and **Average RT**.

---

1. & 2. Priority Scheduling (Non-preemptive & Preemptive)

Concept: Processes are scheduled based on their priority. Higher priority processes (usually indicated by a lower priority number) execute before lower priority processes.

- **Non-preemptive:** Once a process starts executing, it runs to completion (or until it blocks) without interruption, even if a higher priority process arrives.

- **Preemptive:** If a higher priority process arrives while a lower priority process is executing, the running process is interrupted (preempted) and the higher priority process takes over the CPU.

Process Data:

Process	Arrival Time	Burst Time	Priority
P1	52	41	10
P2	0	30	40
P3	30	53	20
P4	25	25	30

Tasks: * Solve for **Non-preemptive Priority Scheduling**. * Solve for **Preemptive Priority Scheduling**.

---

3. HRRN (Highest Response Ratio Next)

Concept: A non-preemptive scheduling algorithm that aims to improve fairness by prioritizing processes based on their "response ratio." It favors shorter jobs and jobs that have been waiting for a long time.

- **Non-preemptive:** Once a process starts, it runs to completion.
- **Response Ratio (RR):** Calculated for each waiting process. The process with the highest RR is chosen next.
  - `Response Ratio = (Waiting Time + Burst Time) / Burst Time`
  - `Waiting Time = Current Time - Arrival Time`

Process Data:

Process	Arrival Time	Burst Time
P1	52	41
P2	0	30
P3	30	53
P4	25	25

Tasks: * Solve using HRRN scheduling.

---

4. Round Robin (RR)

Concept: A preemptive scheduling algorithm designed for time-sharing systems. Each process gets a small, fixed unit of CPU time, called a **Time Quantum (TQ)**. If a process doesn't complete within its quantum, it's preempted and moved to the end of the ready queue.

- **Preemptive:** Processes are interrupted after their time quantum expires.
- **Time Quantum (TQ): 10**

Process Data:

Process	Arrival Time	Burst Time
P1	52	41
P2	0	30
P3	30	53
P4	25	25

Tasks: * Solve using Round Robin scheduling with a Time Quantum of 10.

---

5. Bonus Round: Choose One

Concept: This problem asks you to select one of two basic scheduling algorithms.

- **FCFS (First-Come, First-Served):** Processes are executed in the order they arrive. Non-preemptive.
- **SJF Non-preemptive (Shortest Job First):** The process with the shortest burst time among the available processes is executed next. Once started, it runs to completion. Non-preemptive.

Process Data:

Process	Arrival Time	Burst Time
P1	52	41
P2	0	30
P3	30	53

Process	Arrival Time	Burst Time
P4	25	25

Tasks: * **Choose EITHER FCFS OR SJF Non-preemptive.** * Solve using your chosen algorithm.